Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №3
Варіант 14
На тему
«Незмінні структури даних у Python»

Виконав:
Студент групи ФЕП-21с
Хороший Святослав

Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи

Застосувати tuple, frozenset, dataclasses із frozen=True і MappingProxyType. Замінити мутації створенням нових версій даних та порівняти правильність і час виконання операцій до й після переходу на незмінну модель.

Теоретичні відомості

Tuple зберігає впорядковані незмінні колекції, frozenset — незмінні множини. Декоратор dataclass(frozen=True, slots=True) забороняє звичайне переприсвоєння полів та прибирає словник атрибутів. Вкладені значення також мають бути незмінними.

MappingProxyType дає доступ тільки для читання до словника. Перед обгортанням потрібно скопіювати джерело, щоб зовнішні зміни не впливали на модель. Dataclasses.replace створює новий об’єкт зі зміненими полями. Незмінні частини можна спільно використовувати між версіями.

Хід роботи

Для варіанта 14 я реалізував IoT-сенсори. Reading містить час і значення показу. Sensor містить ідентифікатор, тип, tuple показів, frozenset міток і read-only config. Обидві сутності є frozen dataclass. Конфігурація допускає тільки скалярні незмінні значення.

У core.py я створив add_reading, add_label і update_config. Усі вони повертають новий Sensor. Додатково реалізував чисте калібрування показів і конвеєр рейтингу сенсорів одного типу за середнім значенням. Читання показів і друк розділені: I/O винесено в app.py та benchmark.py.

Reading і Sensor можна використовувати як ключі dict. Поле config виключено з hash, але залишено в порівнянні рівності. Тому різні конфігурації можуть мати однаковий hash; коректність забезпечує подальша перевірка рівності.

Тести перевіряють незмінність, нові копії, read-only config, захист від зміни зовнішнього словника та хешованість. Для порівняння швидкості я створив контрольну мутабельну модель з deepcopy перед append і frozen-модель з tuple та replace. Обидві повертають незалежну версію без зміни оригіналу.

У парних тестах використано 1000 показів і 40 оновлень. Додатковий timeit-замір обчислює медіану п’яти повторів по 100 операцій; підготовку початкових даних у цей замір не включено. Нижче наведено код і фактичні результати.

lab03 / model.py

```python
"""Незмінні сутності IoT-сенсорів, варіант 14."""

from collections.abc import Mapping
from dataclasses import dataclass, field
from math import isfinite
from types import MappingProxyType

ConfigValue = str | int | float | bool


@dataclass(frozen=True, slots=True)
class Reading:
    observed_at: str
    value: float

    def __post_init__(self) -> None:
        if not isfinite(self.value):
            raise ValueError("Показ має бути скінченним числом")


@dataclass(frozen=True, slots=True)
```

Рис. 1. Файл model.py, рядки 1–21

lab03 / model.py

```python
class Sensor:
    id: str
    kind: str
    readings: tuple[Reading, ...] = ()
    labels: frozenset[str] = frozenset()
    config: Mapping[str, ConfigValue] = field(default_factory=dict, hash=False)

    def __post_init__(self) -> None:
        # Копіюємо джерело: proxy не відображатиме зовнішніх змін.
        config = dict(self.config)
        if any(
            not isinstance(key, str) or type(value) not in (str, int, float, bool)
            for key, value in config.items()
        ):
            raise TypeError("config допускає лише рядкові ключі та скалярні значення")
        if any(isinstance(v, float) and not isfinite(v) for v in config.values()):
            raise ValueError("Числові параметри config мають бути скінченними")
        object.__setattr__(self, "readings", tuple(self.readings))
        object.__setattr__(self, "labels", frozenset(self.labels))
        object.__setattr__(self, "config", MappingProxyType(config))
```

Рис. 2. Файл model.py, рядки 22–41

lab03 / core.py

```python
"""Чисті оновлення, обчислення та конвеєр незмінних сенсорів."""

from collections.abc import Mapping, Sequence
from dataclasses import replace
from math import isfinite

from model import ConfigValue, Reading, Sensor


def add_reading(sensor: Sensor, reading: Reading) -> Sensor:
    return replace(sensor, readings=sensor.readings + (reading,))


def add_label(sensor: Sensor, label: str) -> Sensor:
    return replace(sensor, labels=sensor.labels | frozenset({label}))


def update_config(sensor: Sensor, updates: Mapping[str, ConfigValue]) -> Sensor:
    return replace(sensor, config={**sensor.config, **updates})


def average_reading(sensor: Sensor) -> float:
    if not sensor.readings:
        return 0.0
    return sum(reading.value for reading in sensor.readings) / len(sensor.readings)
```

Рис. 3. Файл core.py, рядки 1–25

lab03 / core.py

```python


def calibrate(sensor: Sensor, *, factor: float, offset: float) -> Sensor:
    """Нова версія показів: value * factor + offset."""
    if not isfinite(factor) or not isfinite(offset):
        raise ValueError("Параметри калібрування мають бути скінченними")
    readings = tuple(
        Reading(r.observed_at, r.value * factor + offset) for r in sensor.readings
    )
    return replace(sensor, readings=readings)


def rank_sensors(
    sensors: Sequence[Sensor], *, kind: str, min_readings: int, top_n: int
) -> tuple[Sensor, ...]:
    """Конвеєр: один тип сенсорів → достатньо показів → середнє → Top-N."""
    if min_readings < 0 or top_n < 0:
        raise ValueError("min_readings і top_n мають бути невід'ємними")
    accepted = filter(
        lambda s: s.kind == kind and len(s.readings) >= min_readings, sensors
    )
    scored = map(lambda s: (s, average_reading(s)), accepted)
    ranked = sorted(scored, key=lambda pair: (-pair[1], pair[0].id))
    return tuple(sensor for sensor, _ in ranked[:top_n])
```

Рис. 4. Файл core.py, рядки 26–49

lab03 / app.py

```
"""Демонстраційна I/O-оболонка для варіанта 14."""

from core import (
    add_label,
    add_reading,
    average_reading,
    calibrate,
    rank_sensors,
    update_config,
)
from model import Reading, Sensor


def main() -> None:
    original = Sensor(
        id="T-01",
        kind="temperature",
        readings=(Reading("09:00", 20.0), Reading("10:00", 22.0)),
        labels=frozenset({"indoor"}),
        config={"unit": "C", "interval": 60},
    )
    updated = update_config(
        add_label(add_reading(original, Reading("11:00", 24.0)), "verified"),
        {"interval": 30},
    )
    calibrated = calibrate(updated, factor=1.0, offset=0.5)
    print("=== IOT-СЕНСОРИ | ВАРІАНТ 14 ===")
    for title, sensor in [
        ("Оригінал", original),
```

Рис. 5. Файл app.py, рядки 1–29

lab03 / app.py

```
        ("Нова версія", updated),
        ("Калібрування", calibrated),
    ]:
        print(
            f"{title}: показів={len(sensor.readings)}, середнє={average_reading(sensor):.2f}"
        )
    print(f"Мітки оригіналу: {', '.join(sorted(original.labels))}")
    print(f"Нові мітки: {', '.join(sorted(updated.labels))}")
    print(f"Інтервал: {original.config['interval']} -> {updated.config['interval']}")
    print(f"Новий об'єкт: {original is not updated}")
    print(
        f"Sensor як ключ dict: {{original: 'saved'}} -> { {original: 'saved'}[original] }"
    )
    try:
        updated.config["interval"] = 1  # type: ignore[index]
    except TypeError:
        print("config: зміну заблоковано (TypeError)")
    top = rank_sensors(
        (original, Sensor("T-02", "temperature", (Reading("10:00", 23.0),))),
        kind="temperature",
        min_readings=1,
        top_n=2,
    )
    print("Рейтинг сенсорів:", ", ".join(s.id for s in top))


if __name__ == "__main__":
    main()
```

Рис. 6. Файл app.py, рядки 30–57

lab03 / baseline.py

```python
"""Мутабельна модель «до» для чесного порівняння з immutable API."""

from copy import deepcopy
from typing import TypedDict

from model import ConfigValue


class MutableReading(TypedDict):
    observed_at: str
    value: float


class MutableSensor(TypedDict):
    id: str
    kind: str
    readings: list[MutableReading]
    labels: set[str]
    config: dict[str, ConfigValue]


def add_reading_copy(sensor: MutableSensor, reading: MutableReading) -> MutableSensor:
    """Захисна копія перед append зберігає початковий мутабельний стан."""
    copied = deepcopy(sensor)
    copied["readings"].append(deepcopy(reading))
    return copied
```

Рис. 7. Файл baseline.py, рядки 1–26

lab03 / benchmark.py

```python
"""Оболонка вимірювань: медіани timeit, без порогів швидкості в тестах."""

from statistics import median
from timeit import repeat

from baseline import MutableSensor, add_reading_copy
from core import add_reading
from model import Reading, Sensor


def main() -> None:
    size, number, repeats = 1000, 100, 5
    immutable = Sensor(
        "T-01",
        "temperature",
        tuple(Reading(str(i), float(i)) for i in range(size)),
        frozenset({"indoor"}),
        {"unit": "C"},
    )
    mutable: MutableSensor = {
        "id": immutable.id,
        "kind": immutable.kind,
        "readings": [
            {"observed_at": r.observed_at, "value": r.value} for r in immutable.readings
        ],
        "labels": set(immutable.labels),
        "config": dict(immutable.config),
    }
    reading = Reading("new", 25.0)
    before = (
        median(
```

Рис. 8. Файл benchmark.py, рядки 1–31

lab03 / benchmark.py

```
            repeat(
                lambda: add_reading_copy(
                    mutable, {"observed_at": "new", "value": 25.0}
                ),
                number=number,
                repeat=repeats,
            )
        )
        / number
    )
    after = (
        median(
            repeat(
                lambda: add_reading(immutable, reading),
                number=number,
                repeat=repeats,
            )
        )
        / number
    )
    print(f"Показів: {size}; повторів: {repeats}; операцій на повтор: {number}")
    print("Порівняння: нова незалежна версія, без зміни оригіналу")
    print(f"До: mutable + deepcopy + append: {before * 1e6:.2f} мкс/операцію")
    print(f"Після: frozen + tuple + replace: {after * 1e6:.2f} мкс/операцію")
    print(f"Відношення часу до/після: {before / after:.2f}")
    print("Це локальний вимір, а не гарантія швидкості для інших даних.")


if __name__ == "__main__":
    main()
```

Рис. 9. Файл benchmark.py, рядки 32–61

lab03 / tests/test_immutability.py

```
from collections.abc import Mapping
from dataclasses import FrozenInstanceError, replace
from types import MappingProxyType
from typing import cast

import pytest

from core import (
    add_label,
    add_reading,
    average_reading,
    calibrate,
    rank_sensors,
    update_config,
)
from model import ConfigValue, Reading, Sensor


@pytest.fixture
def sensor() -> Sensor:
    return Sensor(
        "T-01",
        "temperature",
        (Reading("09:00", 20.0), Reading("10:00", 22.0)),
        frozenset({"indoor"}),
        {"unit": "C", "interval": 60},
    )


def test_add_reading_is_pure(sensor: Sensor) -> None:
    original_values = tuple(r.value for r in sensor.readings)
    reading = Reading("11:00", 24.0)
    result = add_reading(sensor, reading)
    assert result is not sensor
```

Рис. 10. Файл tests/test_immutability.py, рядки 1–34

lab03 / tests/test_immutability.py

```
35      assert tuple(r.value for r in sensor.readings) == original_values
36      assert result.readings == sensor.readings + (reading,)
37      assert average_reading(result) == 22
38      assert add_reading(sensor, reading) == result
39
40
41  def test_add_label_is_pure(sensor: Sensor) -> None:
42      result = add_label(sensor, "verified")
43      assert result is not sensor
44      assert sensor.labels == frozenset({"indoor"})
45      assert result.labels == frozenset({"indoor", "verified"})
46      assert add_label(result, "verified") == result
47
48
49  def test_config_update_is_pure(sensor: Sensor) -> None:
50      updates: dict[str, ConfigValue] = {"interval": 30, "enabled": True}
51      result = update_config(sensor, updates)
52      updates["interval"] = 999
53      assert sensor.config["interval"] == 60
54      assert result.config["interval"] == 30
55      assert result.config["unit"] == "C"
56      assert result.config["enabled"] is True
57
58
59  def test_source_config_is_copied() -> None:
60      source: dict[str, ConfigValue] = {"unit": "C"}
61      sensor = Sensor("T-01", "temperature", config=source)
62      source["unit"] = "changed"
63      assert isinstance(sensor.config, MappingProxyType)
64      assert sensor.config["unit"] == "C"
65
66
67  def test_read_only_config(sensor: Sensor) -> None:
68      with pytest.raises(TypeError):
```

Рис. 11. Файл tests/test_immutability.py, рядки 35–68

lab03 / tests/test_immutability.py

```
69          sensor.config["unit"] = "F"  # type: ignore[index]
70
71
72  def test_frozen_entities(sensor: Sensor) -> None:
73      with pytest.raises(FrozenInstanceError):
74          sensor.id = "changed"  # type: ignore[misc]
75      with pytest.raises(FrozenInstanceError):
76          sensor.readings[0].value = 99  # type: ignore[misc]
77      assert not hasattr(sensor, "__dict__")
78
79
80  def test_hashability_and_equality(sensor: Sensor) -> None:
81      equal = replace(sensor, config=dict(sensor.config))
82      changed = add_reading(sensor, Reading("11:00", 24))
83      assert sensor == equal and hash(sensor) == hash(equal)
84      assert {sensor: "saved"}[equal] == "saved"
85      assert len({sensor, equal, changed}) == 2
86      # config враховано в рівності, але виключено з hash.
87      different = update_config(sensor, {"unit": "F"})
88      assert different != sensor
89      assert hash(different) == hash(sensor)
90      assert len({sensor, different}) == 2
91      assert len({Reading("09:00", 20), Reading("09:00", 20)}) == 1
92
93
94  def test_calibration_returns_new_readings(sensor: Sensor) -> None:
95      result = calibrate(sensor, factor=2, offset=1)
96      assert [r.value for r in result.readings] == [41, 45]
97      assert [r.value for r in sensor.readings] == [20, 22]
98      assert result.readings[0] is not sensor.readings[0]
99      assert result.readings[0].observed_at == "09:00"
100
101
102 def test_rank_and_tie_breaker(sensor: Sensor) -> None:
```

Рис. 12. Файл tests/test_immutability.py, рядки 69–102

lab03 / tests/test_immutability.py

```
    tied = replace(sensor, id="T-00")
    other_kind = replace(sensor, id="H-01", kind="humidity")
    assert rank_sensors(
        [sensor, tied, other_kind],
        kind="temperature",
        min_readings=2,
        top_n=1,
    ) == (tied,)
    assert sensor.id == "T-01"


def test_empty_cases() -> None:
    empty = Sensor("T-01", "temperature")
    assert average_reading(empty) == 0
    assert rank_sensors([empty], kind="temperature", min_readings=1, top_n=3) == ()
    assert rank_sensors([empty], kind="temperature", min_readings=0, top_n=0) == ()


@pytest.mark.parametrize("value", [float("nan"), float("inf"), -float("inf")])
def test_non_finite_reading(value: float) -> None:
    with pytest.raises(ValueError):
        Reading("09:00", value)


def test_invalid_config_and_parameters(sensor: Sensor) -> None:
    bad = cast(Mapping[str, ConfigValue], {"nested": [1, 2]})
    with pytest.raises(TypeError):
        Sensor("T-01", "temperature", config=bad)
    with pytest.raises(ValueError):
        calibrate(sensor, factor=float("nan"), offset=0)
    with pytest.raises(ValueError):
        rank_sensors([sensor], kind="temperature", min_readings=1, top_n=-1)
```

Рис. 13. Файл tests/test_immutability.py, рядки 103–134

lab03 / tests/test_performance.py

```
"""Парні навантажувальні тести, час порівнюється через --durations=5."""

from baseline import MutableSensor, add_reading_copy
from core import add_reading
from model import Reading, Sensor


def test_before_mutable_defensive_copy() -> None:
    sensor: MutableSensor = {
        "id": "T-01",
        "kind": "temperature",
        "readings": [{"observed_at": str(i), "value": float(i)} for i in range(1000)],
        "labels": {"indoor"},
        "config": {"unit": "C"},
    }
    for _ in range(40):
        result = add_reading_copy(sensor, {"observed_at": "new", "value": 25.0})
        assert len(result["readings"]) == 1001
```

Рис. 14. Файл tests/test_performance.py, рядки 1–18

lab03 / tests/test_performance.py

```
        assert result["readings"][-1]["value"] == 25.0
    assert len(sensor["readings"]) == 1000


def test_after_immutable_shared_readings() -> None:
    sensor = Sensor(
        "T-01",
        "temperature",
        tuple(Reading(str(i), float(i)) for i in range(1000)),
        frozenset({"indoor"}),
        {"unit": "C"},
    )
    for _ in range(40):
        result = add_reading(sensor, Reading("new", 25.0))
        assert len(result.readings) == 1001
        assert result.readings[-1].value == 25.0
        assert result.readings[0] is sensor.readings[0]
    assert len(sensor.readings) == 1000
```

Рис. 15. Файл tests/test_performance.py, рядки 19–36

Запуск і результати перевірки

lab03 / Вивід команд терміналу

```
$ python app.py
=== IOT-СЕНСОРИ | ВАРІАНТ 14 ===
Оригінал: показів=2, середнє=21.00
Нова версія: показів=3, середнє=22.00
Калібрування: показів=3, середнє=22.50
Мітки оригіналу: indoor
Нові мітки: indoor, verified
Інтервал: 60 -> 30
Новий об'єкт: True
Sensor як ключ dict: {original: 'saved'} -> saved
config: зміну заблоковано (TypeError)
Рейтинг сенсорів: T-02, T-01
```

Рис. 16. Запуск демонстрації

lab03 / Вивід команд терміналу

```
$ python -m pytest -q --durations=5 --durations-min=0
.............                                                            [100%]
============================== slowest 5 durations ==============================
0.03s call     tests/test_performance.py::test_before_mutable_defensive_copy
0.00s call     tests/test_performance.py::test_after_immutable_shared_readings
0.00s setup    tests/test_immutability.py::test_add_reading_is_pure
0.00s call     tests/test_immutability.py::test_add_reading_is_pure
0.00s setup    tests/test_immutability.py::test_non_finite_reading[nan]
16 passed in 0.03s

$ python -m mypy .
Success: no issues found in 7 source files

$ python -m black --check .
All done! ✨ 🍰 ✨
7 files would be left unchanged.

$ python -m ruff check .
All checks passed!
```

Рис. 17. Тести, тривалість і перевірки якості

Порівняння часу операцій

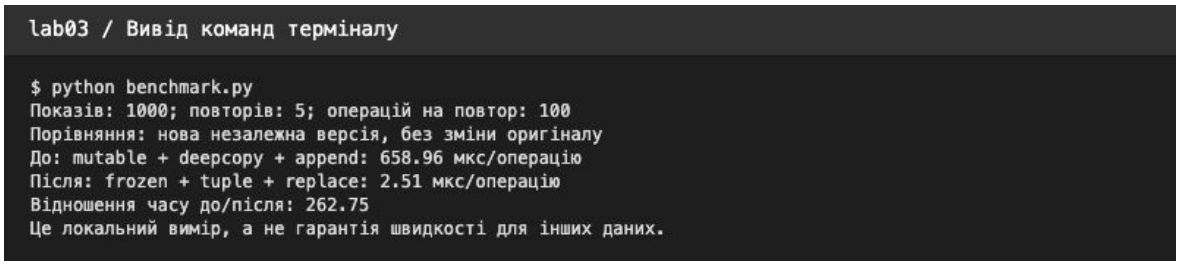


Рис. 18. Порівняння часу операцій

Порівнюються два способи створення незалежної версії сенсора. Мутабельна модель копіює вкладені словники через deepcopy. Незмінна модель створює tuple з посиланнями на попередні frozen Reading і новий Sensor через replace. Ця різниця в обсязі копіювання пояснює отримані результати.

Висновок

Я створив незмінну модель IoT-сенсорів і замінив зміни стану pure-оновленнями. Реалізував додавання показу, мітки та зміну конфігурації, калібрування й рейтинг. Початкові сенсори залишаються незмінними, конфігурація доступна тільки для читання. Усі 16 тестів пройшли; mypy, black і ruff помилок не виявили.

Незмінність спрощує зберігання попередніх версій та тестування. Спільні незмінні покази зменшують обсяг захисного копіювання. Водночас конкатенація tuple потребує O(n) копіювання посилань, а нові об’єкти й копія config створюють накладні витрати. Наведений локальний вимір характеризує саме цей сценарій і не доводить універсального прискорення.

Роботу виконано за Lab_03.docx, варіант 14. Структуру файлів адаптовано зі зразка github.com/r1tokk/lab03.